



FloraFetch

Online Plant Marketplace Platform

Phase 1 — Project Proposal & Technical Documentation

Course	CS519 — Web Engineering
Semester	Spring 2026
Phase	Phase 1: Project Proposal
Supervisor	Muhammad Hassaan
Supervisor Email	m.hassaan@vu.edu.pk
Submitted By	Muhammad Asim
Date	25 April 2026

Virtual University of Pakistan
Department of Computer Science

1. Executive Summary

FloraFetch is a full-stack, web-based e-commerce platform designed for the online sale and delivery of live plants, succulents, herbs, and gardening essentials. The platform addresses a growing market gap in Pakistan's urban gardening sector — where no centralized, trustworthy digital marketplace exists for plant enthusiasts and local nurseries alike.

This document constitutes the Phase 1 submission for CS519 (Spring 2026). It covers the complete project proposal, system scope, functional and non-functional requirements, chosen technology stack with justification, database design, system architecture, and project timeline.

FloraFetch will be built using React.js for the frontend, Node.js/Express for the backend REST API, and SQLite as the database engine — a lightweight yet powerful combination that enables rapid development, clean component architecture, and easy deployment suitable for an academic project of this scope.

2. Project Overview

2.1 Problem Statement

Urban gardening and plant parenting have seen rapid growth in Pakistan, particularly in cities like Lahore, Islamabad, and Karachi. Despite this trend, plant buyers face several persistent challenges:

- No reliable online platform exists that specializes in live plant delivery
- Local nurseries have no digital presence and cannot reach customers beyond their immediate vicinity
- Buyers have no way to verify plant health, care requirements, or authenticity before purchase
- Order tracking and post-delivery support are entirely absent from the current market

2.2 Proposed Solution

FloraFetch solves these problems by providing a dedicated, plant-focused e-commerce platform. Key capabilities include:

- A curated, categorized plant catalog with detailed care guides per listing
- Real-time inventory management for nursery administrators
- A streamlined Cash on Delivery (COD) checkout to build buyer trust
- End-to-end order tracking from nursery preparation to doorstep delivery
- A community review system where buyers share plant arrival photos and ratings

2.3 Project Scope

The following modules are in scope for this project:

1. User Registration, Authentication & Profile Management
2. Admin Dashboard — Inventory, Orders & User Management
3. Plant Catalog — Categories, Filters & Product Detail Pages
4. Shopping Cart & Checkout Flow

5. Cash on Delivery Payment Confirmation
6. Order Tracking System (4-stage pipeline)
7. User Reviews & Admin Moderation

The following items are explicitly out of scope:

- Online payment gateway integration (Easypaisa, JazzCash, Stripe)
- Mobile application (iOS / Android)
- Third-party delivery partner API integration

3. Technology Stack & Justification

The FloraFetch platform uses a modern, industry-standard JavaScript-based stack. Every technology choice is deliberate and justified below.

Layer	Technology	Version	Purpose
Frontend	React.js	v18.x	UI component library — SPA architecture
Frontend	React Router	v6.x	Client-side routing & navigation
Frontend	Axios	v1.x	HTTP requests to backend REST API
Frontend	CSS Modules	Built-in	Scoped, maintainable component styling
Backend	Node.js	v20 LTS	JavaScript runtime — server environment
Backend	Express.js	v4.x	REST API framework — routing & middleware
Backend	JWT	v9.x	Stateless authentication tokens
Backend	bcrypt	v5.x	Password hashing & security
Backend	Multer	v1.x	Image upload handling (plant photos)
Database	SQLite	v3.x	Lightweight relational database
Database	better-sqlite3	v9.x	Node.js SQLite driver (synchronous)
Dev Tools	Vite	v5.x	Frontend build tool & dev server
Dev Tools	Nodemon	v3.x	Auto-reload Node.js on code changes
Dev Tools	Postman	Latest	API testing & documentation

3.1 Why React.js?

React.js was selected as the frontend framework for the following reasons:

- **Component-Based Architecture:** Every UI element (ProductCard, CartItem, OrderTracker) is a reusable component, reducing code duplication and improving maintainability.
- **Single Page Application (SPA):** React enables seamless navigation without full page reloads, giving FloraFetch a fast, app-like user experience essential for an e-commerce platform.
- **React Router v6:** Handles all client-side routes (/products, /cart, /admin/dashboard) cleanly and supports protected routes for admin-only pages.
- **State Management:** React's built-in useState and useContext hooks manage cart state, user session, and product filters without needing a heavy external library.
- **Ecosystem:** A massive library ecosystem (React Query, Axios, React Hook Form) provides ready-made solutions for data fetching, form handling, and caching.

- **Industry Relevance:** React is the most widely used frontend framework (2024 Stack Overflow Survey), making this project directly aligned with industry practice.

3.2 Why SQLite?

SQLite was chosen as the database for FloraFetch over alternatives like MySQL or PostgreSQL for the following reasons:

- **Zero Configuration:** SQLite requires no server installation, no network configuration, and no service management — a single .db file IS the database. This is ideal for academic development.
- **Fully Relational:** Despite its simplicity, SQLite supports all standard SQL operations — JOINS, transactions, foreign keys, indexes — everything FloraFetch needs.
- **Portability:** The entire database is a single file that can be version-controlled, shared, and deployed alongside the application code with zero setup on any machine.
- **Performance:** For a platform of this scale (hundreds to low thousands of records), SQLite performs as well as full RDBMS solutions, with read speeds often exceeding MySQL in single-user contexts.
- **better-sqlite3 Driver:** The synchronous Node.js driver eliminates callback complexity, making database queries in Express routes clean and readable.

3.3 Why Node.js + Express?

- **Unified Language:** JavaScript on both frontend (React) and backend (Node.js) means one language across the entire stack, reducing context switching and enabling shared validation logic.
- **Express.js:** Minimal, unopinionated framework — perfect for building a clean RESTful API with full control over middleware, routing, and error handling.
- **Rich Middleware Ecosystem:** JWT authentication, file uploads (Multer), CORS handling, and request logging are all available as production-tested Express middleware packages.
- **Async Performance:** Node.js's non-blocking I/O model handles concurrent requests efficiently, making it ideal for an e-commerce API that serves multiple users simultaneously.

4. System Architecture

4.1 High-Level Architecture

FloraFetch follows a standard three-tier client-server architecture:

Tier	Technology	Responsibility
Presentation Tier	React.js (Vite dev server)	All UI rendering, user interaction, client routing, cart state
Application Tier	Node.js + Express REST API	Business logic, authentication, data validation, API endpoints
Data Tier	SQLite (better-sqlite3)	Persistent storage of users, products, orders, reviews

4.2 Application Flow

The data flow between layers follows a strict request-response cycle:

8. User interacts with a React component (e.g., clicks 'Add to Cart')
9. React dispatches an Axios HTTP request to the Express REST API
10. Express middleware validates the JWT token and sanitizes the request body
11. The route handler executes a SQL query against the SQLite database
12. The database returns query results to Express
13. Express formats a JSON response and sends it back to React
14. React updates component state and re-renders the UI accordingly

4.3 Folder Structure

Path	Description
florafetch/	Root project directory
frontend/	React application (Vite)
src/components/	Reusable UI components (Navbar, ProductCard, etc.)
src/pages/	Route-level pages (Home, Shop, Cart, Admin, etc.)
src/context/	React Context providers (AuthContext, CartContext)
src/api/	Axios API service functions
backend/	Node.js / Express application
routes/	API route files (auth, products, orders, reviews)
controllers/	Business logic handlers for each route
middleware/	JWT auth middleware, error handling
db/	SQLite database file + schema.sql
uploads/	Stored plant image files (via Multer)

5. Database Design (SQLite)

The FloraFetch database is a relational SQLite database consisting of 7 interconnected tables. All foreign key constraints are enforced. The schema is designed to normalize data, prevent redundancy, and support all functional requirements listed in the project brief.

5.1 Entity Relationship Summary

Table	Primary Key	Description
users	user_id (INT)	Stores registered customers and admin accounts
plants	plant_id (INT)	Full plant catalog with metadata, care info, stock
categories	category_id (INT)	Plant categories (Indoor, Outdoor, Succulents, etc.)
cart_items	cart_item_id (INT)	Temporary cart records linked to user and plant
orders	order_id (INT)	Completed orders with delivery address and status
order_items	item_id (INT)	Individual plants within each order (quantity, price)
reviews	review_id (INT)	Customer reviews with rating, photo, and admin status

5.2 Detailed Table Schemas

Table 1: users

Column	Data Type	Constraints	Description
user_id	INTEGER	PK, AUTOINCREMENT	Unique user identifier
full_name	TEXT	NOT NULL	User's full name
email	TEXT	UNIQUE, NOT NULL	Email address (login credential)
phone	TEXT	UNIQUE	Phone number (optional login)
password	TEXT	NOT NULL	bcrypt hashed password
role	TEXT	DEFAULT 'customer'	Role: 'customer' or 'admin'
addresses	TEXT		JSON string of saved delivery addresses
created_at	DATETIME	DEFAULT NOW	Account creation timestamp

Table 2: plants

Column	Data Type	Constraints	Description
plant_id	INTEGER	PK, AUTOINCREMENT	Unique plant identifier
category_id	INTEGER	FK → categories	Linked category
name	TEXT	NOT NULL	Common plant name

botanical_name	<i>TEXT</i>		Scientific/botanical name
description	<i>TEXT</i>		Full care description
price	<i>REAL</i>	NOT NULL	Price in PKR
size	<i>TEXT</i>		small / medium / large
stock_qty	<i>INTEGER</i>	DEFAULT 0	Current inventory count
sunlight_req	<i>TEXT</i>		Low / Medium / High
watering_freq	<i>TEXT</i>		Daily / Weekly / Fortnightly
is_pet_friendly	<i>INTEGER</i>	DEFAULT 0	Boolean: 0=No, 1=Yes
is_low_maint	<i>INTEGER</i>	DEFAULT 0	Boolean: low maintenance flag
image_url	<i>TEXT</i>		Path to uploaded plant photo
created_at	<i>DATETIME</i>	DEFAULT NOW	Listing creation timestamp

Table 3: orders

Column	Data Type	Constraints	Description
order_id	<i>INTEGER</i>	PK, AUTOINCREMENT	Unique order identifier
user_id	<i>INTEGER</i>	FK → users	Customer who placed the order
delivery_address	<i>TEXT</i>	NOT NULL	Full delivery address string
delivery_date	<i>DATE</i>		Requested delivery date
special_instr	<i>TEXT</i>		Handling instructions for delicate plants
total_amount	<i>REAL</i>	NOT NULL	Total order amount in PKR
payment_method	<i>TEXT</i>	DEFAULT 'COD'	Payment type (Cash on Delivery)
status	<i>TEXT</i>	DEFAULT 'Confirmed'	Order Status (see tracking stages)
created_at	<i>DATETIME</i>	DEFAULT NOW	Order placement timestamp

Table 4: reviews

Column	Data Type	Constraints	Description
review_id	<i>INTEGER</i>	PK, AUTOINCREMENT	Unique review identifier
user_id	<i>INTEGER</i>	FK → users	Reviewer
plant_id	<i>INTEGER</i>	FK → plants	Reviewed plant
order_id	<i>INTEGER</i>	FK → orders	Linked purchase order
rating	<i>INTEGER</i>	CHECK 1-5	Star rating 1 to 5
review_text	<i>TEXT</i>		Written review body
photo_url	<i>TEXT</i>		Uploaded plant-in-home photo
is_approved	<i>INTEGER</i>	DEFAULT 0	Admin moderation status

created_at	<i>DATE TIME</i>	DEFAULT NOW	Review submission timestamp
------------	------------------	-------------	-----------------------------

6. Functional Requirements

6.1 User Registration & Account Management

- Sign-Up / Login / Logout: Secure JWT-based authentication via email or phone number
 - Passwords stored as bcrypt hashes (salt rounds: 12)
 - JWT tokens expire after 24 hours; refresh handled via localStorage
- Profile Management: Users can manage personal details and save multiple named delivery addresses
- Plant History: Previous purchases displayed on the user profile page
- Admin Dashboard: Admins access a protected /admin route with full control over users, inventory, and orders

6.2 Plant & Product Management

- Add / Edit / Delete Products (Admin only): Plant listings include botanical name, price, size, sunlight requirements, watering frequency, pet-friendly flag, and low-maintenance flag
- Categories & Filters: Plants organized under Indoor, Outdoor, Succulents, Flowering, and Medicinal categories
 - Filters: Low Maintenance, Pet Friendly, Price Range, Growth Rate
- Plant Care Page: Each listing has a dedicated detail page with high-quality images, soil recommendations, temperature guidance, and 'frequently bought with' suggestions
- Real-Time Stock Tracking: Stock quantity decrements upon order placement and is visible in the admin inventory panel

6.3 Shopping Cart & Checkout

- Cart Functionality: Add/remove plants and tools, adjust quantities, view subtotal ('Green Total')
- Checkout Flow: Delivery date selection, special handling instructions for delicate plants, address selection from saved addresses
- Order Confirmation: Automated confirmation message displayed immediately after order placement

6.4 Payment — Cash on Delivery

- COD only: Users pay on physical receipt and inspection of the live plant
- No online payment processing required in this phase
- Order receipt generated and stored in the orders table upon checkout completion

6.5 Order Tracking

Orders progress through a 4-stage pipeline:

#	Stage	Description
1	Order Confirmed	Order received by system; confirmation sent to customer

2	Plant Selection/Quality Check	Nursery staff selects and inspects the plant for travel health
3	In Transit	Plant packaged and handed to delivery; tracking active
4	Delivered	Plant received by customer; review prompt triggered

6.6 Feedback & Care Community

- User Reviews: Customers upload photos of their plants at home and rate plant health on arrival (1–5 stars)
- Admin Moderation: Admins approve/reject reviews before they go public and can respond with expert advice

7. Non-Functional Requirements

Requirement	Detail
Security	All passwords hashed with bcrypt. API routes protected by JWT middleware. Admin routes require role verification. Input sanitized to prevent SQL injection.
Performance	Page load target < 2 seconds on standard broadband. React lazy loading for non-critical components. SQLite queries indexed on frequently filtered columns (category_id, plant_id, user_id).
Usability	Mobile-responsive layout using CSS media queries. Minimum touch target size 44x44px. All forms include real-time validation feedback.
Reliability	SQLite transactions ensure data integrity during order placement. Backend error handling returns meaningful HTTP status codes (400, 401, 404, 500).
Maintainability	Component-based React architecture. RESTful API with separation of routes, controllers, and middleware. Documented API endpoints via Postman collection.
Scalability	SQLite is suitable for the current academic scope. Architecture is designed so the database layer can be swapped to PostgreSQL with minimal refactoring if needed.

8. REST API Design

All API endpoints follow RESTful conventions. The base URL for all routes is: `/api/v1`

8.1 Authentication Endpoints

Method	Endpoint	Auth	Description
POST	<code>/api/v1/auth/register</code>	Public	Register a new customer account
POST	<code>/api/v1/auth/login</code>	Public	Login — returns JWT token
POST	<code>/api/v1/auth/logout</code>	JWT	Invalidate current session token
GET	<code>/api/v1/auth/profile</code>	JWT	Get logged-in user profile
PUT	<code>/api/v1/auth/profile</code>	JWT	Update profile / delivery addresses

8.2 Plant Endpoints

Method	Endpoint	Auth	Description
GET	<code>/api/v1/plants</code>	Public	Get all plants (supports filter query params)
GET	<code>/api/v1/plants/:id</code>	Public	Get single plant detail by ID
GET	<code>/api/v1/plants/category/:id</code>	Public	Get plants by category
POST	<code>/api/v1/plants</code>	Admin	Add a new plant listing
PUT	<code>/api/v1/plants/:id</code>	Admin	Update an existing plant listing
DELETE	<code>/api/v1/plants/:id</code>	Admin	Remove a plant listing

8.3 Order & Cart Endpoints

Method	Endpoint	Auth	Description
GET	<code>/api/v1/cart</code>	JWT	Get current user's cart items
POST	<code>/api/v1/cart</code>	JWT	Add item to cart
DELETE	<code>/api/v1/cart/:itemId</code>	JWT	Remove item from cart
POST	<code>/api/v1/orders</code>	JWT	Place a new order (checkout)
GET	<code>/api/v1/orders</code>	JWT	Get all orders for logged-in user
GET	<code>/api/v1/orders/:id</code>	JWT	Track a specific order by ID
PUT	<code>/api/v1/orders/:id/status</code>	Admin	Update order status (admin only)

9. UI Pages & React Components

9.1 Page List

Page	Route	Key Components
Home / Landing	/	HeroBanner, FeaturedPlants, CategoryGrid, Testimonials
Shop / Catalog	/shop	FilterSidebar, ProductGrid, ProductCard, PaginationBar
Plant Detail	/plant/:id	PlantGallery, CareGuide, AddToCartBtn, RelatedItems
Shopping Cart	/cart	CartTable, CartItem, OrderSummary, CheckoutBtn
Checkout	/checkout	AddressSelector, DeliveryDatePicker, OrderReview, PlaceOrderBtn
Order Tracking	/orders/:id	OrderStatusStepper, OrderItemList, DeliveryInfo
User Profile	/profile	ProfileForm, AddressManager, PurchaseHistory
Login / Register	/login, /register	AuthForm, ValidationMessages
Admin Dashboard	/admin	SalesStats, LowStockAlert, RecentOrders
Admin — Inventory	/admin/plants	PlantTable, AddPlantModal, EditPlantForm, DeleteConfirm
Admin — Orders	/admin/orders	OrderTable, StatusDropdown, OrderDetail
Admin — Reviews	/admin/reviews	ReviewQueue, ApproveBtn, ReplyForm

10. Project Timeline & Milestones

Phase	Week(s)	Milestone	Deliverables
1	Week 1–2	Project Proposal	This document — scope, tech stack, DB schema, API design, timeline
2	Week 3–4	Backend Foundation	Node/Express project setup, SQLite schema, auth endpoints, JWT middleware
3	Week 5–6	Core API Development	Plant, category, cart, order, and review API endpoints fully tested
4	Week 7–8	React Frontend — Auth & Catalog	Login/Register pages, Shop page, Plant Detail page, routing setup
5	Week 9–10	React Frontend — Commerce	Cart, Checkout, Order Tracking pages, COD order flow
6	Week 11	Admin Panel	Admin Dashboard, Inventory management, Order status update, Review moderation
7	Week 12	Integration & Testing	Frontend-backend integration, end-to-end testing, bug fixing
8	Week 13	Polish & Submission	UI polish, documentation, demo video, final submission

11. Risk Assessment & Mitigation

Risk	Impact	Mitigation Strategy
SQLite concurrency limits	Medium	SQLite is single-writer; for academic scope this is acceptable. WAL mode enabled to improve read concurrency.
JWT token expiry issues	Low	Implement token refresh logic and clear localStorage on 401 responses to avoid silent auth failures.
Image upload size / storage	Medium	Limit uploads to 2MB via Multer configuration. Store images in /uploads folder with UUID filenames.
React state management complexity	Medium	Use React Context for global state (cart, auth). Avoid prop drilling. Refactor to Zustand if state grows complex.
Scope creep beyond Phase 1	High	Strictly follow the functional requirements listed. Defer online payments, mobile app, and live chat to future phases.
CORS issues between React and API	Low	Configure Express CORS middleware with explicit allowed origins from day one.

12. Conclusion

FloraFetch is a well-scoped, technically sound web application that addresses a genuine market need. The chosen technology stack — React.js, Node.js/Express, and SQLite — provides a modern, industry-relevant foundation that is simultaneously practical for academic development and scalable for future real-world deployment.

The database schema is fully normalized, covers all functional requirements, and uses SQLite's relational capabilities (foreign keys, constraints, indexes) appropriately. The REST API is designed following RESTful best practices with clear separation of public, authenticated, and admin-only routes.

This Phase 1 documentation establishes a clear blueprint for the development phases ahead. All functional and non-functional requirements are traceable to specific technical decisions in the stack, database, and API design sections above.

Supervisor: Muhammad Hassaan — m.hassaan@vu.edu.pk

Course: CS519 — Web Engineering | Spring 2026

Platform: Virtual University of Pakistan

Tech Stack: React.js + Node.js + Express.js + SQLite

Database: SQLite v3 with better-sqlite3 Node.js driver